Déploiement automatisé d’une infrastructure cloud avec Terraform

Rapport de projet

Azure • Terraform • Ubuntu • Node.js/Express • MongoDB • Azure Blob Storage

Étudiante	Sirine EL GAIED
Cloud retenu	Microsoft Azure
Système d’exploitation	Ubuntu 22.04 LTS
Technologies utilisées	Terraform, Node.js/Express, MongoDB, Azure Blob Storage
Dépôt GitHub	https://github.com/sirine1711/teraformer

Ce rapport présente la mise en place d’une infrastructure cloud sur Azure à l’aide de Terraform, puis le déploiement d’un backend Node.js/Express connecté à une base MongoDB et à un stockage Azure Blob. Le scénario de départ citait Flask comme exemple, mais l’énoncé autorisait une autre technologie backend ; le choix retenu ici a donc été Node.js, cohérent avec les outils effectivement utilisés pendant le projet.

1. Contexte et objectifs

L'objectif du projet était d'automatiser la création d'une infrastructure cloud complète à l'aide de Terraform. L'infrastructure devait contenir au minimum une machine virtuelle et un espace de stockage cloud.

Le projet a été réalisé sur Microsoft Azure. L'application déployée est un backend Express exposé sur une VM Ubuntu, capable de communiquer avec MongoDB pour les données persistantes et avec Azure Blob Storage pour la gestion des fichiers.

2. Architecture retenue

L'architecture finale repose sur une VM Ubuntu accessible par IP publique. Le backend Express écoute sur le port 3000, interagit avec MongoDB pour le CRUD et stocke/relit des fichiers dans un conteneur Azure Blob.

Client / navigateur	VM Ubuntu Azure	Base de données	Stockage cloud
Accès HTTP via IP publique Tests avec PowerShell / curl	Node.js + Express Port 3000 Connexion SSH	MongoDB Base projetcloud Collections users/files	Storage Account Azure Container Blob : fichiers

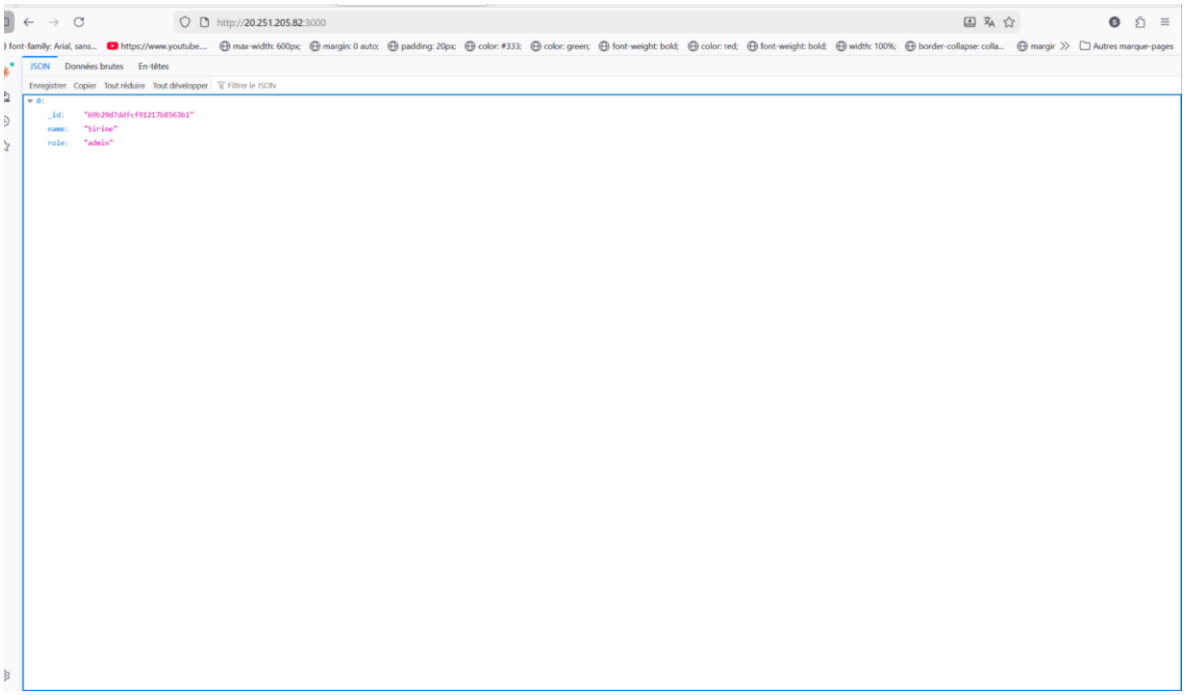

Figure 1 — Réponse du backend depuis le navigateur via l'IP publique de la VM.

3. Étape 1 — Préparer l'environnement Terraform

La première étape a consisté à préparer le projet Terraform sur le poste local, puis à créer les fichiers attendus (provider.tf, main.tf, variables.tf et outputs.tf). Le provider choisi est azurerm pour Azure.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Installez la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations ! https://aka.ms/PSWindow
s

PS C:\Users\sirin> mkdir projet-terraform


    Répertoire : C:\Users\sirin


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        12/03/2026     10:03                projet-terraform


PS C:\Users\sirin> cd projet-terraform
PS C:\Users\sirin\projet-terraform> notepad provider.tf
PS C:\Users\sirin\projet-terraform> notepad main.tf
PS C:\Users\sirin\projet-terraform> notepad variables.tf
PS C:\Users\sirin\projet-terraform> notepad outputs.tf
PS C:\Users\sirin\projet-terraform>
```

Figure 2 — Création du dossier de projet et des fichiers Terraform initiaux.

```
PS C:\Users\sirin\projet-terraform> terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/azurerm versions matching "~> 3.0"...
- Installing hashicorp/azurerm v3.117.1...
- Installed hashicorp/azurerm v3.117.1 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
PS C:\Users\sirin\projet-terraform> terraform plan

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
  + create

Terraform will perform the following actions:

  # azurerm_resource_group.rg will be created
  + resource "azurerm_resource_group" "rg" {
      + id       = (known after apply)
      + location = "francecentral"
      + name     = "rg-terraform-projet"
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + resource_group_name = "rg-terraform-projet"
─────────────────────────────────────────────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if
you run "terraform apply" now.
PS C:\Users\sirin\projet-terraform> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # azurerm_resource_group.rg will be created
```

Figure 3 — Initialisation de Terraform et premier plan d'exécution.

```
Windows PowerShell

Terraform will perform the following actions:

  # azurerm_resource_group.rg will be created
  + resource "azurerm_resource_group" "rg" {
      + id       = (known after apply)
      + location = "francecentral"
      + name     = "rg-terraform-projet"
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + resource_group_name = "rg-terraform-projet"

──────────────────────────────────────────────────────────────────────────────────────────────────────────────────

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly these actions if
you run "terraform apply" now.
PS C:\Users\sirin\projet-terraform> terraform apply

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # azurerm_resource_group.rg will be created
  + resource "azurerm_resource_group" "rg" {
      + id       = (known after apply)
      + location = "francecentral"
      + name     = "rg-terraform-projet"
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  + resource_group_name = "rg-terraform-projet"

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

azurerm_resource_group.rg: Creating...
azurerm_resource_group.rg: Creation complete after 9s [id=/subscriptions/a3efadf7-1c39-43e6-a775-2c6a6859e826/resourceGroups/rg-terraform-projet]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

resource_group_name = "rg-terraform-projet"
PS C:\Users\sirin\projet-terraform>
```

Figure 4 — Premier apply validant la configuration de base du projet.

4. Étape 2 — Déployer l'infrastructure

L'infrastructure principale a été créée sur Azure avec Terraform : groupe de ressources, réseau virtuel, sous-réseau, interface réseau, IP publique, règle de sécurité, machine virtuelle Linux et compte de stockage Azure.

Le système d'exploitation déployé sur la machine virtuelle est Ubuntu 22.04 LTS. Une clé SSH RSA a été utilisée afin d'assurer l'accès à distance à la VM.

```
Windows PowerShell

Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Installez la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations ! https://aka.ms/PSWindow
s

PS C:\Users\sirin> cd C:\Users\sirin\projet-terraform
PS C:\Users\sirin\projet-terraform> dir


    Répertoire : C:\Users\sirin\projet-terraform


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        12/03/2026     10:09                .terraform
-a----        12/03/2026     10:09           1186 .terraform.lock.hcl
-a----        12/03/2026     11:20           3382 main.tf
-a----        12/03/2026     10:17            337 outputs.tf
-a----        12/03/2026     10:53            125 provider.tf
-a----        12/03/2026     11:21          28586 terraform.tfstate
-a----        12/03/2026     10:35          12705 terraform.tfstate.1773308127.backup
-a----        12/03/2026     10:35           5834 terraform.tfstate.1773308137.backup
-a----        12/03/2026     11:09          20755 terraform.tfstate.1773310194.backup
-a----        12/03/2026     11:21          25395 terraform.tfstate.backup
-a----        12/03/2026     10:30            125 variables.tf


PS C:\Users\sirin\projet-terraform>
```

Figure 5 — Dossier Terraform final contenant les fichiers principaux du projet.

```
azureuser@vm-projet: ~/app
  "version": "1.0.0",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^5.2.1"
  },
  "devDependencies": {},
  "description": ""
}


azureuser@vm-projet:~/app$ npm install express

up to date, audited 66 packages in 707ms

22 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
azureuser@vm-projet:~/app$ nano server.js
azureuser@vm-projet:~/app$ node server.js
Server running on port 3000
```

Figure 6 — Installation d'Express sur la VM et lancement initial du backend sur le port 3000.

5. Étape 2 (suite) — Base de données et backend

Le backend retenu est Node.js avec Express. Ce choix est conforme à l'énoncé, qui autorisait un backend autre que Flask. Le serveur a été déployé sur la VM Ubuntu puis exposé sur le port 3000.

Pour la base de données, MongoDB a été installé sur la VM. Une base nommée projetcloud a été créée avec une collection users pour stocker les données de test.

```
mongosh mongodb://127.0.0.
             └─3540 /usr/bin/mongod --config /etc/mongod.conf

Mar 12 10:50:33 vm-projet systemd[1]: Started MongoDB Database Server.
Mar 12 10:50:33 vm-projet mongod[3540]: {"t":{"$date":"2026-03-12T10:50:33.461Z"},"s":"I",  "c":"CONTROL",  "id":748450>
azureuser@vm-projet:~$ mongosh
Current Mongosh Log ID: 69b29afadfcf91217b8563b0
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2
.7.0
Using MongoDB:          7.0.30
Using Mongosh:          2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.co
m/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-03-12T10:50:33.888+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. S
ee http://dochub.mongodb.org/core/prodnotes-filesystem
   2026-03-12T10:50:34.152+00:00: Access control is not enabled for the database. Read and write access to data and conf
iguration is unrestricted
   2026-03-12T10:50:34.152+00:00: For customers running MongoDB 7.0, we suggest changing the contents of the following s
ysfsFile
   2026-03-12T10:50:34.152+00:00: vm.max_map_count is too low
------

test>
```

Figure 7 — Connexion à MongoDB avec mongosh avant création et manipulation des données.

6. Étape 3 — Connecter le backend au stockage cloud et ajouter un CRUD

Le backend a ensuite été relié à Azure Blob Storage grâce à la chaîne de connexion du Storage Account et au conteneur fichiers.

L'API Express propose des routes pour lire et écrire des données utilisateur dans MongoDB, ainsi que pour envoyer et lister des fichiers dans Azure Blob Storage.

Méthode	Route	Rôle
GET	/users	Lister les utilisateurs stockés dans MongoDB
POST	/users	Créer un utilisateur
PUT	/users/:name	Modifier un utilisateur existant
DELETE	/users/:name	Supprimer un utilisateur
POST	/upload	Envoyer un fichier vers Azure Blob
GET	/files	Lister les fichiers présents dans le conteneur Blob

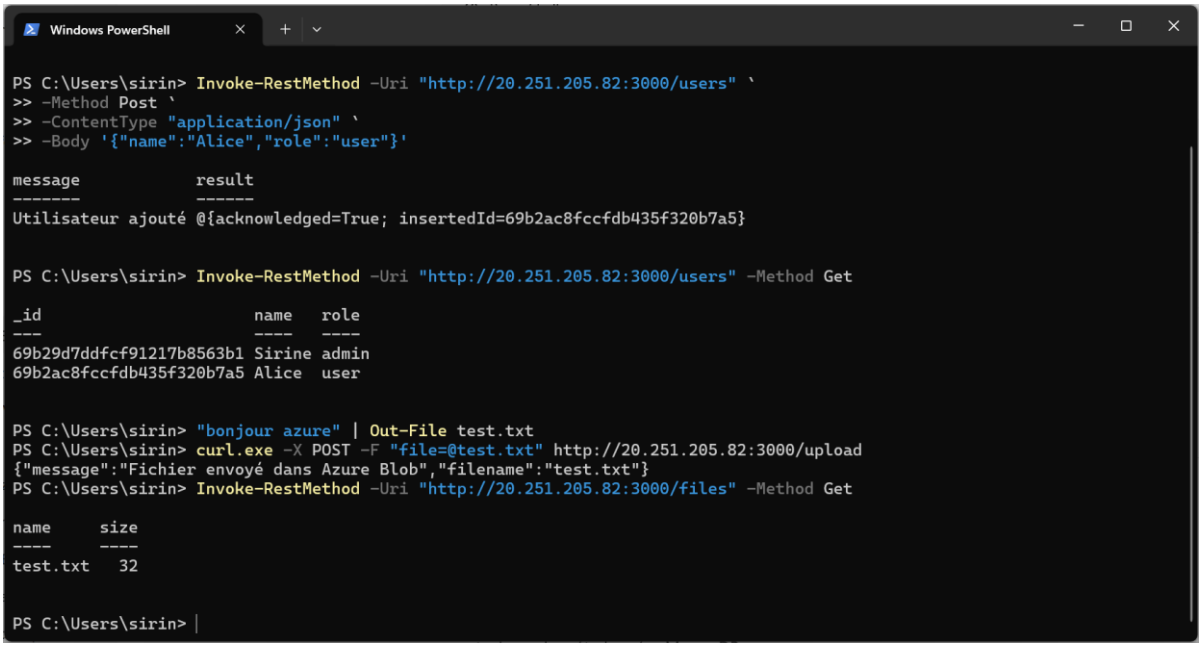

Figure 8 — Tests API : création d'un utilisateur, vérification de la liste, upload d'un fichier puis lecture des fichiers du conteneur.

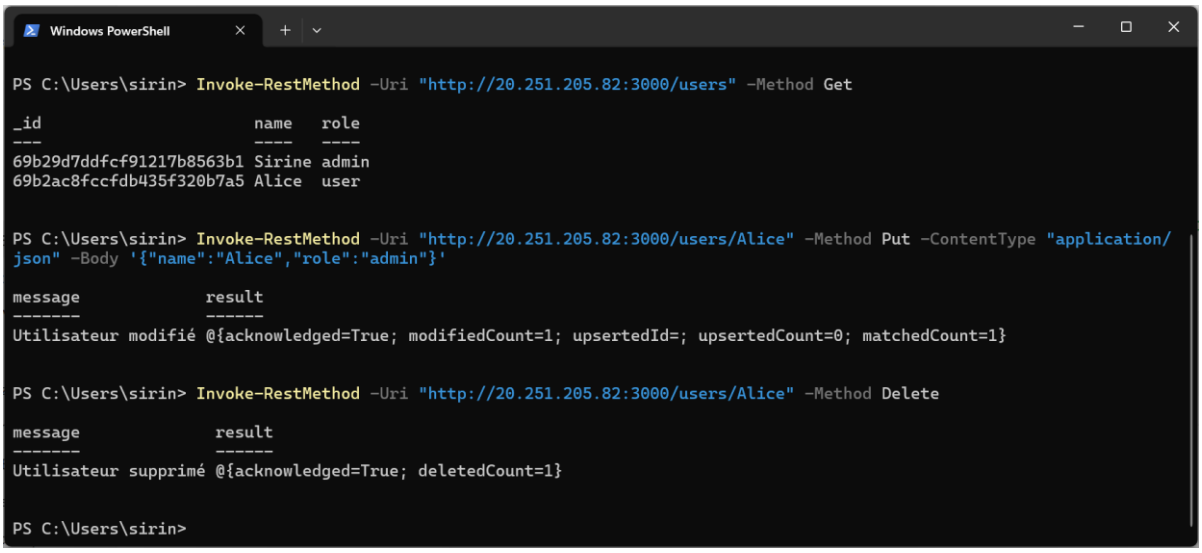

Figure 9 — Validation des opérations CRUD : lecture, modification puis suppression d'un utilisateur.

7. Étape 4 — Automatiser le déploiement avec Terraform

Le projet a été structuré avec variables.tf, terraform.tfvars et outputs.tf afin de rendre le déploiement plus réutilisable et lisible. Les outputs permettent notamment de récupérer l'IP publique de la VM et les identifiants utiles du stockage.

Dans la pratique, l'automatisation la plus robuste obtenue pendant ce projet concerne surtout l'infrastructure Azure. La configuration applicative (Node.js, MongoDB, dépendances npm et backend Express) a été reproduite et validée directement sur la VM à l'aide d'une séquence de commandes claire et répétable. Une amélioration naturelle serait de pousser cette partie encore plus loin via cloud-init, remote-exec, PM2 ou un service systemd.

8. Étape 5 — Tester et détruire l'infrastructure

Les tests fonctionnels ont confirmé l'accès à l'application via l'IP publique, l'exécution des opérations CRUD sur MongoDB et le bon transfert des fichiers vers Azure Blob Storage.

La suppression de l'infrastructure doit ensuite être réalisée depuis le dossier Terraform avec la commande terraform destroy. Cette étape permet de libérer toutes les ressources Azure créées pendant le projet.

```
terraform destroy
```

9. Problèmes rencontrés et solutions apportées

- **Clé SSH non compatible :** Azure refusait la clé ED25519 ; le problème a été résolu en générant une clé RSA 4096 bits puis en l'utilisant dans la ressource Terraform.
- **Indisponibilité de certaines tailles de VM :** Plusieurs tailles de VM n'étaient pas disponibles dans certaines régions Azure ; la région a été ajustée et une taille compatible a été retenue.
- **Ouverture réseau du port 3000 :** L'API n'était pas joignable depuis l'extérieur tant que la règle de sécurité entrante TCP/3000 n'était pas ajoutée dans le groupe de sécurité réseau.

- **Installation de MongoDB sous Ubuntu 22.04 :** Le paquet mongodb n'étant pas disponible directement, le dépôt officiel MongoDB a été ajouté puis le service mongod a été activé.
- **Conflits Node.js / paquets système :** L'installation a nécessité l'usage du dépôt NodeSource et une remise au propre de certains paquets Ubuntu plus anciens.

10. Conclusion

Le projet a permis de déployer sur Azure une infrastructure cloud pilotée par Terraform, puis d'y exécuter un backend Node.js/Express connecté à MongoDB et à Azure Blob Storage.

Les objectifs principaux du sujet ont été atteints : création de la VM, configuration du stockage cloud, déploiement du backend, ajout d'un CRUD simple et validation par des tests concrets via navigateur, PowerShell et curl.

Le dépôt GitHub et les captures sélectionnées constituent une base suffisante pour présenter le projet, justifier les choix techniques et démontrer le bon fonctionnement de la solution.